Tarea 5: Supresión de Ruido de Red Eléctrica

Procesamiento Digital de Señales



Cuatrimestre III - 2026

Tecnológico de Costa Rica
Jafeth Garcia Brenes

Enunciado **3**
Ejercicio 1 3
Ejercicio 2 4
Ejercicio 3 5
Ejercicio 4 6
Ejercicio 5 7
Función de transferencia 7
Polos y ceros 7
Ecuación de diferencias 9
Ejercicio 6 10
Ejercicio 7 11
Conclusiones 12
Bibliografía **12**

Enunciado

En Costa Rica la red eléctrica oscila a una frecuencia de 60Hz. Esta red provee de electricidad a los consumidores, los cuales conectan sus dispositivos electrónicos a la misma para operarlos. Esta red eléctrica alternante, sin embargo, puede causar interferencia y provocar ruido en ciertos circuitos. Lo anterior es especialmente evidente en dispositivos de audio, donde dicha interferencia se percibe como un zumbido audible que coincide con la misma frecuencia (y sus armónicos). Si bien existen configuraciones eléctricas que mitigan estos efectos, en ciertas ocasiones factores económicos o estructurales lo impiden. En otras, la señal de interés simplemente ya ha sido contaminada. Para estos casos, técnicas basadas en Procesamiento Digital de Señales pueden ser utilizadas para reducir este efecto adverso.
Usted ha sido contratado como consultor en PDS para suprimir el ruido causado por la red eléctrica en una señal de audio. Su primera tarea es implementar un prototipo funcional de un sistema que, dada una señal contaminada, produzca una señal donde se ha mitigado considerablemente el efecto de la red.
Provea un script en Python que calcule y despliegue la información requerida por los ítemes siguientes. Para ello haga uso del archivo de audio adjunto a este enunciado denominado la_muerte_del_angel_power_noise.wav. Como referencia, se incluye la misma señal pero sin la interferencia de la línea de poder (la_muerte_del_angel.wav).

Ejercicio 1

¿Cuál es la frecuencia de muestreo con que fue tomada la señal?

Para obtener la frecuencia de muestreo de la señal se abre el archivo .wav y se obtiene la información desde sus metadatos con el código:

```
from scipy.io import wavfile

samplerate, data = wavfile.read('la_muerte_del_angel.wav')

print(f"Sample Rate: {samplerate} Hz")
```

Esto nos devuelve la respuesta de:

R/ Sample Rate: 48000 Hz

Ejercicio 2

Utilice la DFT para desplegar el contenido espectral de la señal en su totalidad. Asegúrese que el eje x de los gráficos tenga las unidades apropiadas en frecuencia (Hz).

Antes de iniciar el procesamiento de la señal se le aplica la normalización a la señal. De los metadatos se tiene 16bps y 1 canal en la señal. Se realiza la normalización con:

$$R = [-2^{m-1}, 2^{m-1} - 2^{-n}].$$

Manteniendo m=1 y n=16 se tiene que el rango sería de R = [-1, 0.99998474121] . Con los datos normalizados se puede proceder a hacer el análisis de la señal.

Se utiliza la FFT que es una optimización para la DFT. Recordando que la DFT utiliza la ecuación:

$$X[k] = \sum_{n=0}^{N-1} x(n)e^{-j2\pi kn/N} \; k = 0, 1, \ldots, N-1$$

Una vez aplicado el algoritmo sobre la señal se obtuvo la magnitud de la señal aplicando para N=963584 (esto equivale a la cantidad de muestras:

$$X[k]_{magn} = |X[k]|$$

En la Fig. 1 se puede ver la gráfica del espectro en magnitud para la señal.

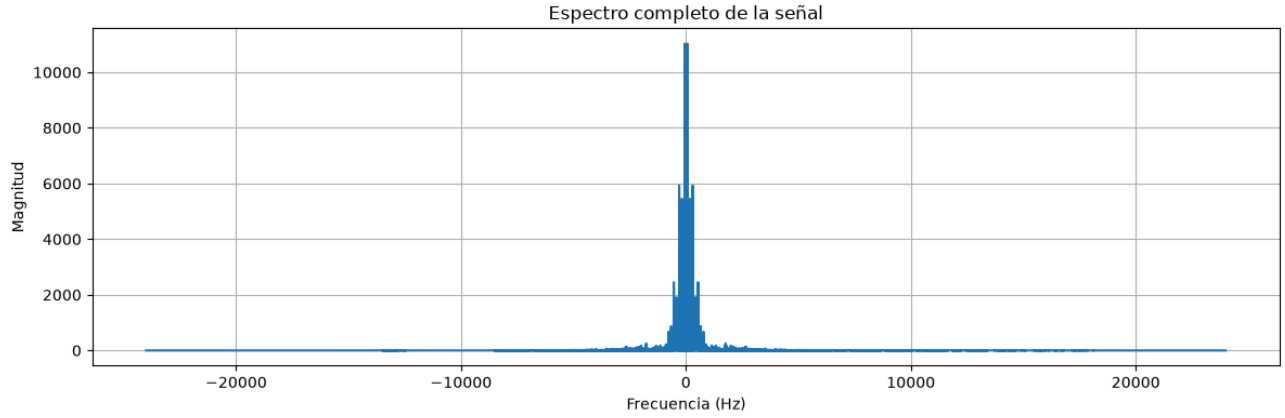


Figura 1. Espectro en magnitud de la señal analizada

Ejercicio 3

Identifique en el espectro las componentes de frecuencia causadas por la línea de poder de 60Hz, así como sus posibles armónicos. Para ello, realice una ampliación del espectro en la región de interés pero asegúrese que las unidades se mantengan correctas.

Tomando en cuenta la frecuencia de 60Hz, cada armónico se presenta en múltiplos de $f_k = n60Hz$.

Realizando la ampliación vista en la Fig. 2, se observa que los armónicos más relevantes son aquellos dentro de los 1000Hz. Por encima de esto, la amplitud decae de una magnitud de 10,000 a ~100.

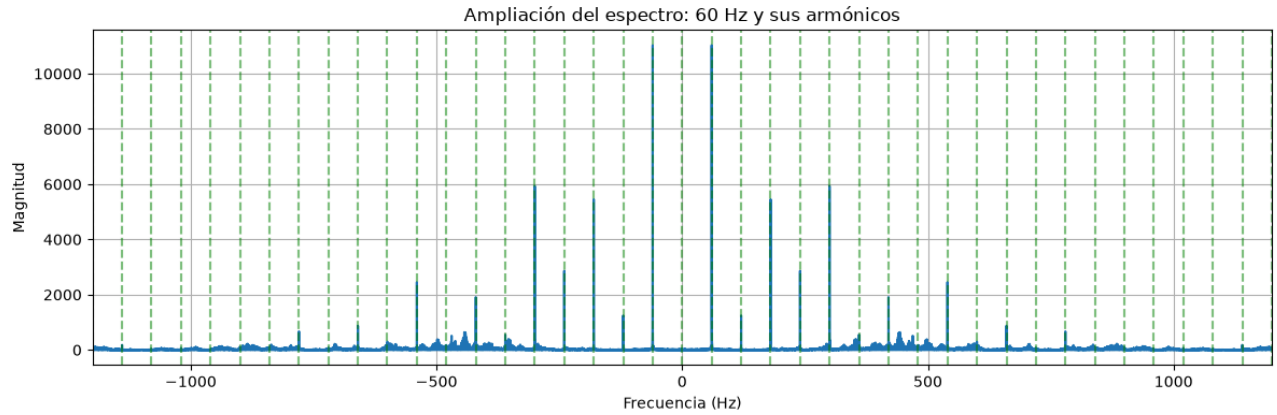


Figura 2. Ampliación del espectro en magnitud con armónicos de 60Hz resaltados

Ejercicio 4

Presente la Transformada Discreta de Fourier de Corto Plazo para 4 ventanas distintas de mucho menor tamaño. Identifique las mismas componentes parásitas en dichos espectros.

La DFT de corto plazo hace referencia a un cambio en la forma de analizar la señal considerando señales que cambian su forma a lo largo del tiempo [1]. Para esto se define la transformada según:

$$X[n,\lambda] = \sum_{m=-\infty}^{\infty} x[n+m]w[m]e^{-j\lambda m}$$

Donde w[n] es una ventana de tiempo, y λ es la frecuencia. Esto se puede interpretarse como la DFT la señal desplazada x[n+m] visto por una ventana de w[m]. Lo cual resulta en extraer diferentes porciones de la señal.

Se tomaron ventanas de 0.5s de la señal original. Sobre estas señales se aplicó el FFT para obtener los diferentes espectros que se observan en la Fig. 3. En estos se observan armónicos de diferentes amplitudes en comparación a la Fig. 2, sin embargo, comparten las frecuencias donde presentaban mayor intensidad.

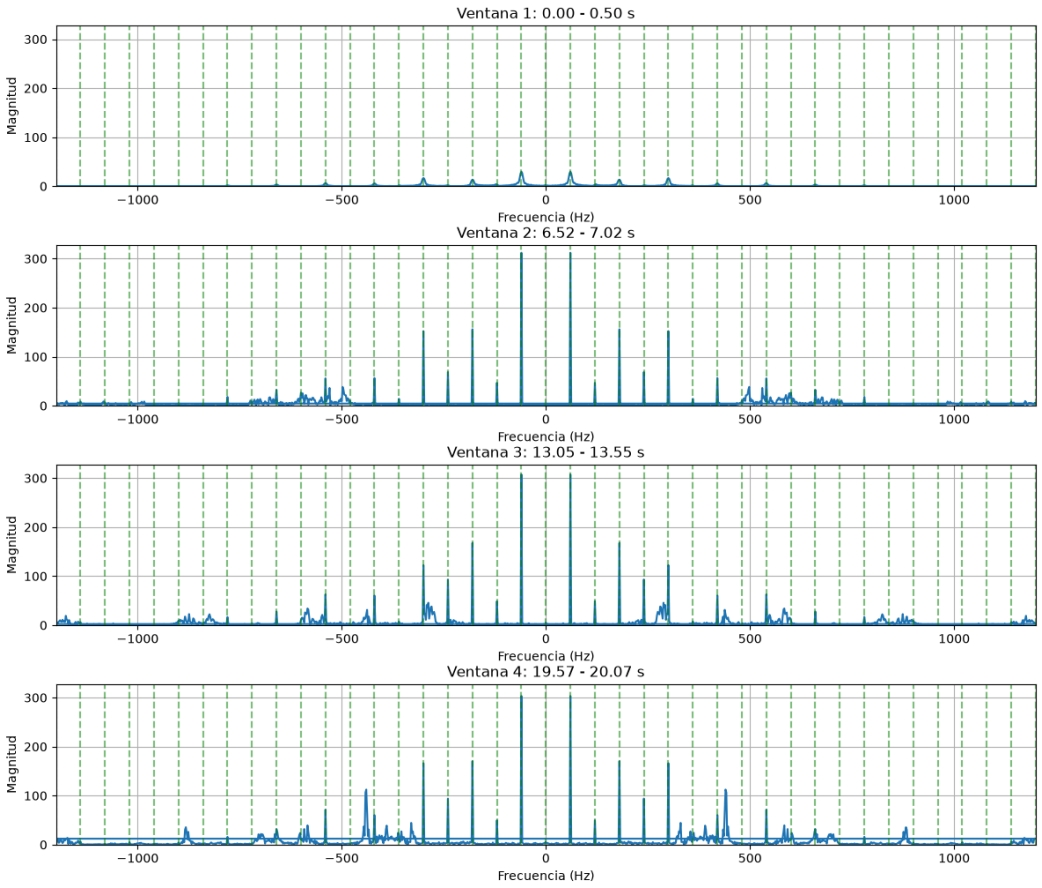


Figura 3. Espectros en magnitud de DFT en corto plazo para segmentos de 0.5s

Ejercicio 5

Proponga un sistema IIR que suprima las componentes de ruido de 60Hz y sus armónicos. Presente su función de transferencia así como su ecuación de diferencias y su diagrama de polos y ceros.

Función de transferencia

Debido a que la señal presenta gran cantidad de armónicos de 60Hz, conviene utilizar un filtro peine para simplificar el diseño. La función de transferencia genérica para un filtro peine IIR se puede expresar como:

$$H(z) = b\frac{1-z^{-N}}{1-az^{-N}}$$

Donde 'b' es un factor de normalización para la ganancia y 'a' es un parámetro de realimentación que debe estar entre $0<a<1$ [2]. Cuando este es 0, se tiene la versión FIR del filtro.

Para que el filtro rechace las frecuencias de 60Hz, se debe seleccionar N de la forma:

$$N = \frac{f_s}{60Hz} = \frac{48000Hz}{60Hz} = 800$$

Para evitar tener ganancia en el filtro, se escoge el factor de normalización según:

$$b = \frac{1+a}{2}$$

Para intentar afectar la menor cantidad de frecuencias, se busca tener una banda de rechazo estrecha. Esto se controla mediante el valor de 'a', al acercarse a 1, más estrecha es la banda.

Seleccionando de forma arbitraria un a=0.9 se tiene que el valor de b sería:

$$b = \frac{1+0.9}{2} = 0.95$$

Esto llega a darnos que la función de transferencia es de:

$$H(z) = 0.95\frac{1-z^{-800}}{1-0.9z^{-800}}$$

Polos y ceros

Para obtener los ceros se iguala a cero el numerador:

$$1 - z^{-800} = 0$$
$$z^{800} = 1$$

Esto significa que los ceros se encuentran dados por:

$$z_k = e^{j2\pi k/800} \quad k = 0, 1, 2, \ldots, 799$$

Para encontrar los polos igualamos el denominador a cero:

$$1 - 0.9z^{-800} = 0$$
$$z^{800} = 0.9$$

Esto hace que los polos sean ubicados en:

$$p_k = \sqrt[800]{0.9}e^{j2\pi k/800} \quad k = 0, 1, 2, \ldots, 799$$

El diagrama como tal se realiza en Python debido a la cantidad de polos existentes. En la Fig. 4 se puede ver el círculo unitario y el diagrama entero. Para dar mejor claridad, se realiza un acercamiento del diagrama en la Fig. 5. Se observa como los polos se encuentran cerca de los ceros pero, si se selecciona un a=1 se cancelan, por esto se establece el límite para el valor de este parámetro.

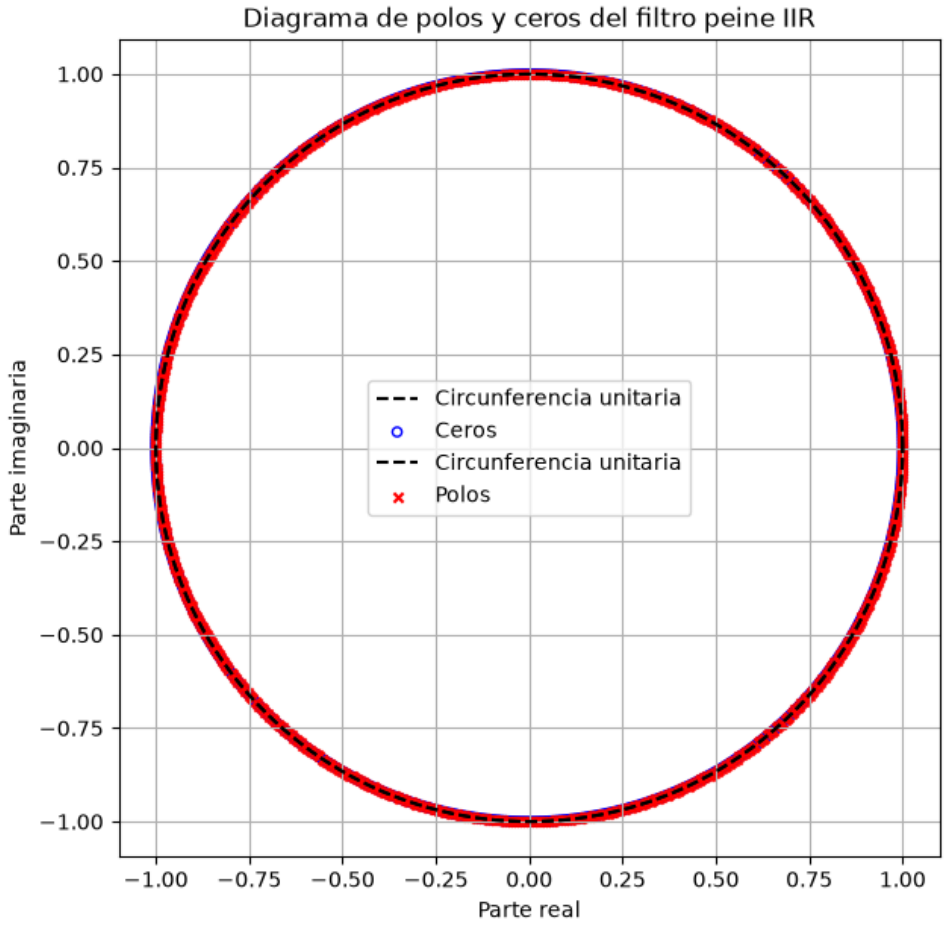


Figura 4. Diagrama de polos y ceros para filtro peine

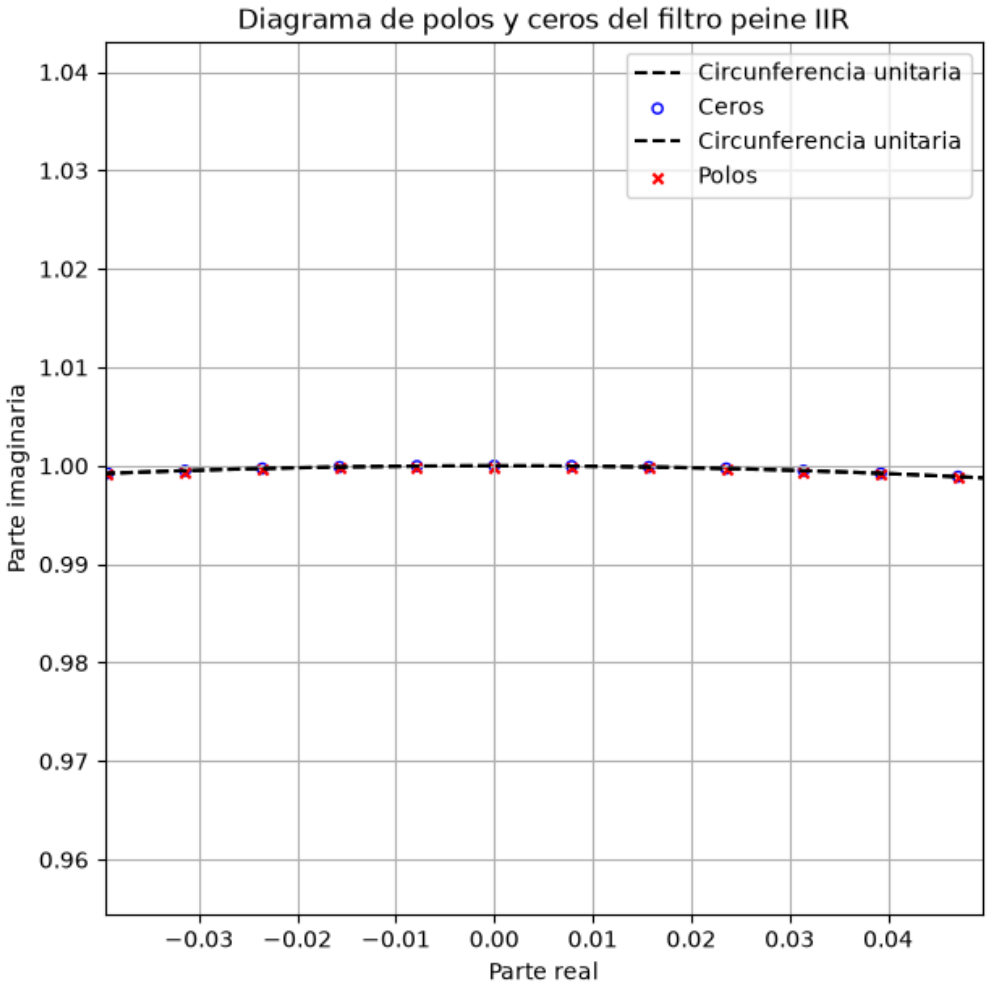


Figura 5. Acercamiento del diagrama de polos y ceros

Ecuación de diferencias

Para obtener la ecuación de diferencias se parte de:

$$\frac{Y(z)}{X(z)} = 0.95\frac{1-z^{-800}}{1-0.9z^{-800}}$$

Multiplicando ambos lados por X(z) y el denominador de la función de transferencia:

$$Y(z) - Y(z)0.9z^{-800} = X(z)0.95 - X(z)0.95z^{-800}$$

Transformada inversa:

$$y[n] - 0.9y[n-800] = 0.95x[n] - 0.95x[n-800]$$

$$y[n] = 0.95x[n] - 0.95x[n-800] + 0.9y[n-800]$$

Ejercicio 6

Despliegue su respuesta con frecuencia. Para ello utilice alguna herramienta como scipy.signal.freqz (o equivalente). Asegúrese que el eje x tenga unidades de Hz.

En la Fig. 6 se observa la magnitud para la respuesta en frecuencia del filtro diseñado. Este seguiría hasta los 24000Hz con el comportamiento mostrado en este acercamiento.

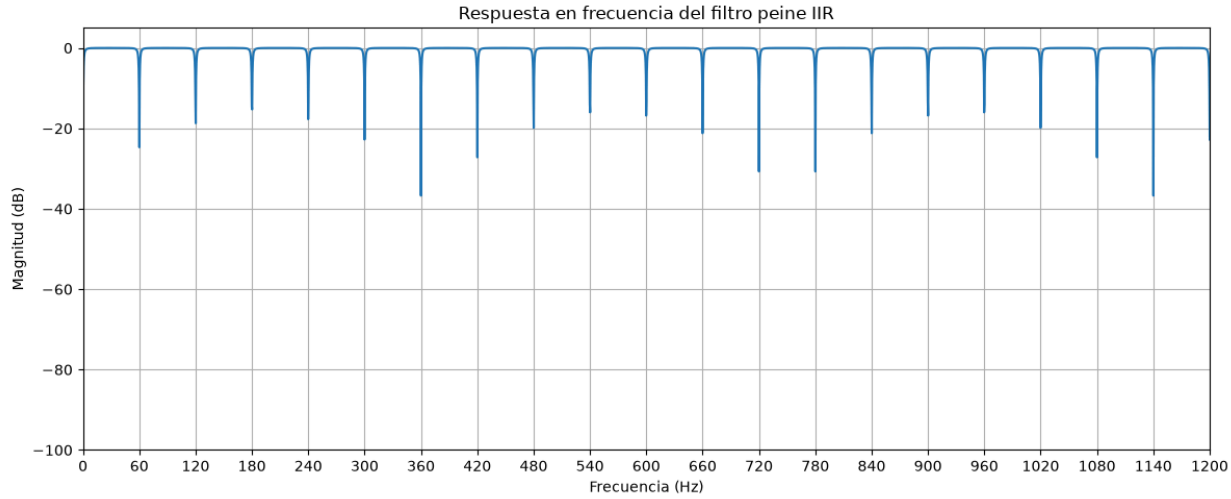


Figura 6. Magnitud(dB) para la respuesta en frecuencia de filtro peine

Ejercicio 7

Filtre la señal contaminada utilizando alguna herramienta como scipy.signal.lfilter (o equivalente). No implemente el filtro manualmente mediante la convolución.

Se realizó el filtrado en el script de Python utilizando scipy.signal.lfilter(). Esto permitió recuperar una señal limpia en la salida audio_filtrado.wav. El código del filtrado se muestra a continuación:

```
fs, audio = wavfile.read('la_muerte_del_angel_power_noise.wav')

# Normalizar a float64
audio_norm = audio.astype(np.float64) / 32768.0
print(fs)



f0 = 60
alpha = 0.9
N = fs // f0
K = (1 + alpha) / 2



# Numerador
b = np.zeros(N + 1)
b[0] = K
b[N] = -K

# Denominador
a = np.zeros(N + 1)
a[0] = 1.0
a[N] = -alpha

# Filtrado
audio_filtrado = lfilter(
   b,
   a,
   audio_norm,
   axis=0
)
```

Se agregaron además funciones para realizar el cálculo de métricas de PSNR, SNR y MSE sobre la señal original y filtrada con respecto a su referencia y se obtuvieron los siguientes resultados.

Nota: Estos cálculos se realizaron sobre las señales después de normalizarlas a Float64

Tabla 1. Métricas de evaluación para señal de audio

Señal	SNR (dB)	PSNR(dB)	MSE
Original	-2.88	32.55	0.00055583
Filtrada	12.25	47.67	0.00001709

Conclusiones

La señal contaminada mostró una mejora significativa después de su filtración. La mejora de SNR de -2.88db a 12.25dB muestra que se logra rescatar la señal. Aunque el filtro puede estar degradando la señal. Si se quisiera mejorar el SNR se podría hacer que el parámetro de retroalimentación ‘*a’* n sea mayor. Esto se debe realizar con cuidado, ya que el filtro puede experimentar efectos de rizado si la frecuencia de corte es muy brusca.
La señal en PSNR y la MSE respaldan la comprobación de la mejora, esta última llegando a tener una disminución en el error de aproximadamente 92%. A pesar de que se puede lograr mejor resultado, se tiene un impacto significativo con el diseño actual.

Bibliografía

[1] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
[2] National Instruments, LabVIEW Digital Filter Design Toolkit User Manual. Austin, TX, USA: National Instruments Corporation, 2005. [En línea]. Disponible en: https://download.ni.com/support/manuals/371353a.pdf